GUIDE DU DÉVELOPPEUR

Créer des mods pour Claude Code

Un guide complet, du premier « bonjour » jusqu'à la publication sur GitHub. Pour tout développeur qui connaît un peu TypeScript, sans connaissance préalable du système de mods.

- Comprendre ce qu'est un mod et comment le moteur l'appelle
- Écrire, charger et recharger un premier mod en dix minutes
- Dessiner une bande, un panneau, des boutons ; réagir aux événements
- Garder des données, ajouter des réglages, piloter Claude
- Tester, déboguer, publier et mettre à jour

Rédigé à partir de Claude Code 2.1.317 et des mods du dépôt **Teddy57600/teddy-mods** (context-weather, usage-limits, usage-forecast, turn-alert). Novembre 2026.
L'API des mods est en accès anticipé : elle peut changer d'une version à l'autre. La référence qui fait foi est le fichier de types fourni par votre propre version (chapitre 16).

Sommaire

1 Qu'est-ce qu'un mod ?
mods, plugins, ce qu'on peut faire

2 Ce qu'il faut installer
Claude Code, éditeur, TypeScript

3 Votre premier mod en 10 minutes
trois fichiers, valider, charger

4 Les concepts essentiels
événements, hooks, `$`, `e`, `next`

5 Anatomie d'un mod
plugin.json, hooks.json, register.tsx

6 Les événements
cycle d'une session et d'un tour

7 Afficher quelque chose
toasts, bandes, panneaux, boutons

8 Garder des données
variables, `$.state`, `$.store`

9 Agir sur Claude
prompts, consignes, outils, garde-fous

10 Commandes, minuteries, système
`/commande`, fichiers, réseau, son

11 Les réglages utilisateur
`userConfig` et `/config`

12 Tester son mod
validate, tests automatiques, tsc

13 Déboguer
messages d'erreur, journal, budget

14 Publier et mettre à jour
marketplace GitHub, versions

15 Pièges et bonnes pratiques
ce que l'expérience a appris

16 Aide-mémoire et références
tableaux récapitulatifs, glossaire

Comment lire ce guide. Les chapitres 1 à 4 se lisent dans l'ordre : ils posent les bases. Ensuite, chaque chapitre répond à une question (« comment afficher… », « comment garder… ») et peut être lu seul. Les exemples de code sont complets et se chargent tels quels.

Conventions. Les blocs sombres contiennent du code TypeScript ; les blocs verts, des commandes à taper dans un terminal. `mon-mod` désigne le dossier (et le nom) de votre mod. Les chemins sont écrits avec `/` ; sous Windows, `~` désigne `C:\Users\<vous>`.

1 Qu'est-ce qu'un mod ?

Claude Code est l'assistant de programmation d'Anthropic. Il fonctionne dans un terminal (la commande `claude`), dans l'application de bureau Claude (onglet Code), dans VS Code et sur mobile. Partout, c'est le même **moteur** qui tourne : il reçoit vos messages, appelle le modèle, exécute les outils (lire un fichier, lancer une commande…) et dessine l'interface.

Un **mod** est un petit programme TypeScript que ce moteur charge et qu'il appelle à des moments précis : quand il dessine une zone de l'écran, quand vous envoyez un message, quand Claude veut lancer une commande, quand un tour de réponse se termine… À chacun de ces moments, votre code peut **observer**, **modifier**, **bloquer** ou **remplacer** ce que le moteur allait faire.

Mod ou plugin ?

Techniquement, un mod est un **plugin** Claude Code. Les plugins peuvent contenir plusieurs choses : des *skills* (instructions que Claude lit), des commandes, des serveurs MCP, des hooks de réglages (scripts shell)… On appelle « mod » un plugin qui contient un **module de hooks** : du code TypeScript exécuté par le moteur lui-même. C'est ce qui permet de dessiner dans l'interface et d'intervenir finement sur chaque étape.

Mécanisme	Langage	Peut dessiner ?	Usage typique
Skill	Markdown	non	apprendre une méthode à Claude
Hook de réglages (`settings.json`)	script shell	non	lancer un formateur après chaque édition
Serveur MCP	tout langage	non	donner de nouveaux outils à Claude
Mod (module de hooks)	**TypeScript / JSX**	**oui**	**tout ce qui précède, plus l'interface**

Ce qu'un mod peut faire

Dans l'interface

- afficher une **bande** au-dessus de la zone de saisie (jauge, compteur, météo…) ;
- ouvrir un **panneau latéral** qui reste à côté de la conversation ;
- montrer un **toast**, une entrée dans la **ligne d'état**, une **notification** système ;
- jouer un **son** ;
- redessiner certaines lignes de la conversation.

Dans le comportement

- réécrire ou compléter un message avant que Claude le lise ;
- ajouter des consignes permanentes au prompt système ;
- **autoriser ou bloquer** un appel d'outil (garde-fou) ;
- offrir de **nouveaux outils** à Claude ;
- ajouter des **commandes** `/mon-mod` ;
- lancer des tâches périodiques, lire des fichiers, appeler un site web.

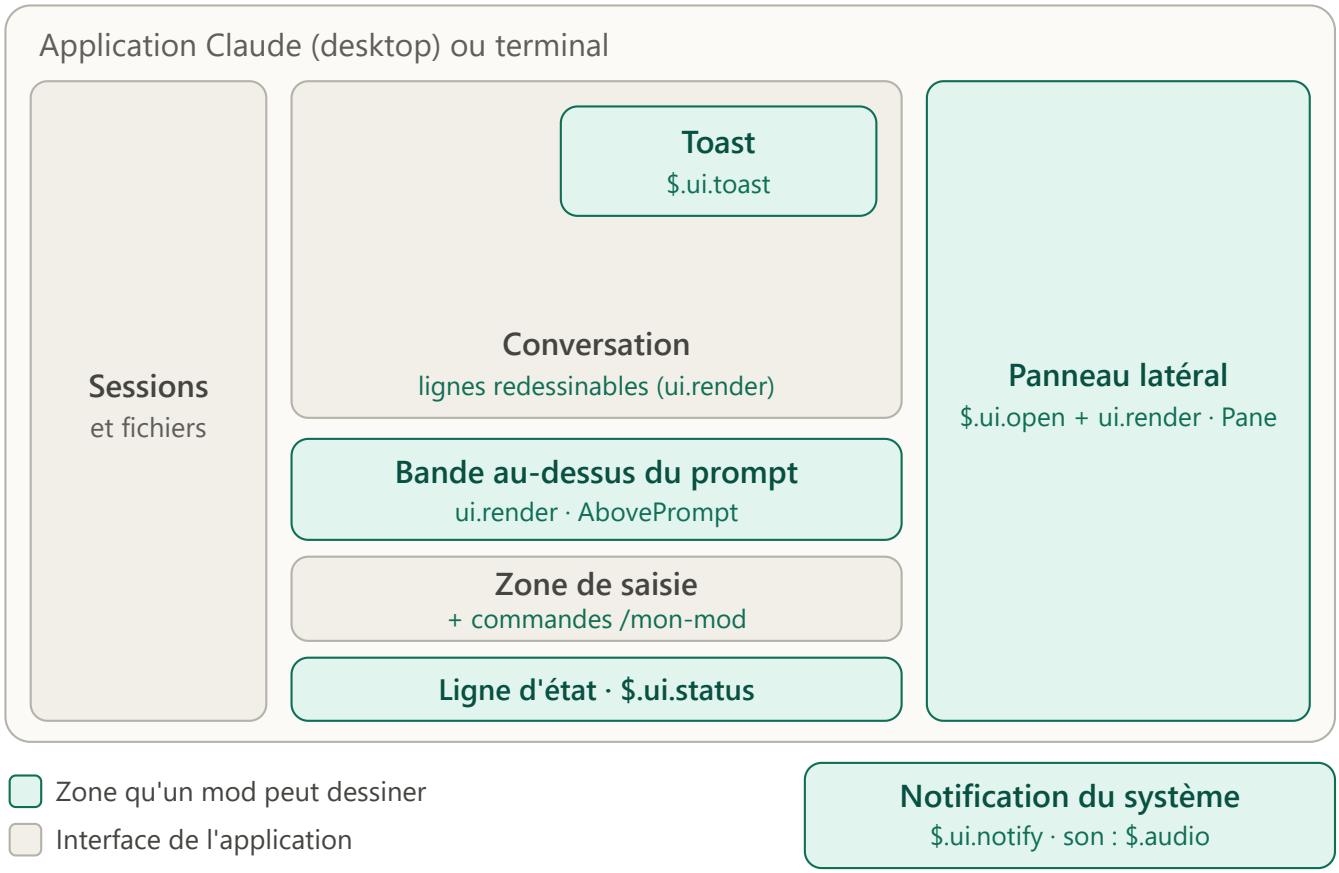


Figure 1 — Les endroits de l'écran où un mod peut afficher quelque chose.

Ce qu'un mod ne peut pas faire

- Il n'a **ni DOM, ni Node.js** : pas de `document`, pas de `require('fs')`, pas de `fetch` direct, pas de `setTimeout`. Tout passe par l'objet `$` que le moteur lui donne (fichiers, réseau, minuteries…).
- Il ne dessine qu'avec les **éléments** que la surface propose (`Box`, `Text`, `Button`…) : pas de HTML ni de CSS libre (sauf dans un `Svg` sur le bureau).
- Il ne peut pas changer ce que le modèle a **déjà reçu** dans une conversation (chapitre 9).

2 Ce qu'il faut installer

Indispensable

- **Claude Code** en ligne de commande (`claude`), version récente. Vérifiez avec :

  ```
  claude --version
  ```

  Sous Windows, l'application de bureau embarque sa propre copie de la CLI, qui n'est pas dans le `PATH` : installez aussi la CLI pour taper les commandes de ce guide.
- Un **éditeur de code** qui comprend TypeScript (VS Code, par exemple). Il vous donnera l'autocomplétion sur tout l'objet `$`.

Utile

- **Git** et un compte **GitHub**, pour partager vos mods (chapitre 14).
- **TypeScript** (`npm install -g typescript`) si vous voulez vérifier les types en dehors de l'éditeur avec `tsc`. Ce n'est pas obligatoire : le moteur compile lui-même vos fichiers.

> **Pas de build, pas de `npm install`.** Un mod n'a pas de `package.json` ni de dépendances : le moteur lit directement vos fichiers `.ts` ou `.tsx`. Le seul import « externe » est `'claude-code'`, fourni par le moteur.

Connaissances supposées

- Les bases de **TypeScript** (ou de JavaScript moderne) : fonctions fléchées, `async` / `await`, déstructuration `{ a, b } = obj`, décomposition `{ ...e, x }`.
- Un peu de **JSX** (la syntaxe `<Box>…</Box>` de React) pour dessiner. Il n'y a pas de React : le JSX sert seulement à écrire des arbres d'éléments lisibles.

3 Votre premier mod en 10 minutes

Objectif : un mod qui affiche un toast « Tour terminé en 12 s » à la fin de chaque réponse de Claude.

Étape 1 — Créer le dossier et trois fichiers

```
bonjour/
├─ .claude-plugin/
│  └─ plugin.json      ← carte d'identité du mod
└─ hooks/
   ├─ hooks.json       ← indique quel fichier contient le code
   └─ register.tsx     ← le code
```

`.claude-plugin/plugin.json`

```
{
  "name": "bonjour",
  "version": "0.1.0",
  "description": "Affiche la durée de chaque tour de Claude"
}
```

`hooks/hooks.json`

```
{ "modules": ["./register.tsx"] }
```

`hooks/register.tsx`

```
import type { Register } from 'claude-code'

export const register: Register = on => {
  // À chaque fin de tour, le moteur appelle cette fonction.
  on('turn.complete', ($, e, next) => {
    const secondes = Math.round(e.durationMs / 1000)
    $.ui.toast(`Tour terminé en ${secondes} s`)
    return next(e)   // laisse le moteur (et les autres mods) continuer
  })
}
```

Étape 2 — Vérifier

```
claude plugin validate bonjour
```

La commande lit le manifeste et le code comme le ferait le moteur, liste ce que le mod branche (ici : `turn.complete`) et signale tout ce qui serait refusé. Lancez-la souvent.

Étape 3 — Charger le mod

```
claude --plugin-dir ./bonjour
```

Une session démarre avec votre mod. Posez une question à Claude : à la fin de sa réponse, le toast apparaît.

Étape 4 — Modifier et voir le changement

Sans quitter la session, changez le texte du toast et enregistrez. Le dossier est **surveillé** : le mod est rechargé tout seul (environ un quart de seconde après la dernière sauvegarde). Au tour suivant, le nouveau texte s'affiche.

> **Encore plus simple : demander à Claude.** Dans une session Claude Code, décrivez le mod voulu (« fais-moi un mod qui affiche un toast avec la durée de chaque tour »). Claude écrit les fichiers dans un dossier de mods de la session et vous demande une fois « Turn on the mods Claude writes in this session? ». Répondez *Turn them on* : le mod se charge à la fin du tour, et chaque modification suivante aussi. C'est un excellent moyen d'apprendre : lisez ensuite le code produit.

Et dans l'application de bureau ?

L'application ne permet pas de passer `--plugin-dir`. Utilisez la variable d'environnement `CLAUDE_CODE_PLUGIN_DIRS`, déclarée dans le bloc `env` de `~/.claude/settings.json` (chemins absolus, séparés par `;` sous Windows et `:` ailleurs) :

```
{
  "env": {
    "CLAUDE_CODE_PLUGIN_DIRS": "C:\\Users\\moi\\mods\\bonjour;C:\\Users\\moi\\mods\\autre"
  }
}
```

Redémarrez l'application : chaque dossier listé est chargé comme avec `--plugin-dir`, et rechargé à chaque sauvegarde.

4 Les concepts essentiels

Événements et hooks

Le moteur émet des **événements** : `session.start`, `prompt.submit`, `tool.call`, `ui.render`, `turn.complete`… Un **hook** est une fonction que vous accrochez à un événement avec `on` :

```
on('tool.call', ($, e, next) => { … })                  // tous les appels d'outils
on('tool.call', { tool: 'Bash' }, ($, e, next) => { … })   // seulement Bash
```

Le deuxième argument facultatif est un **filtre** (*matcher*) : le hook n'est appelé que si l'événement correspond. Il sert aussi au typage : avec `{ tool: 'Bash' }`, l'éditeur sait que `e.command` existe.

Les trois paramètres : `$`, `e`, `next`

`$` — l'interface du moteur
: Tout ce que votre mod peut demander, rangé par « nom » puis « méthode » : `$.ui.toast()`, `$.session.usage()`, `$.store.get()`, `$.fs.read()`, `$.clock.every()`… Votre éditeur les liste tous.

`e` — les données de l'événement
: Un objet figé (on ne le modifie pas) : la commande Bash que Claude veut lancer, le texte que vous avez tapé, la durée du tour… Son contenu dépend de l'événement.

`next` — la suite
: Appeler `next(e)` passe la main aux autres mods puis au comportement normal du moteur, et renvoie le résultat. C'est le cœur du système.

La chaîne : comment `next` fonctionne

Quand plusieurs mods écoutent le même événement, ils forment une **chaîne**, comme des poupées russes. Chacun reçoit l'événement, peut agir avant, appeler la suite, puis agir après :

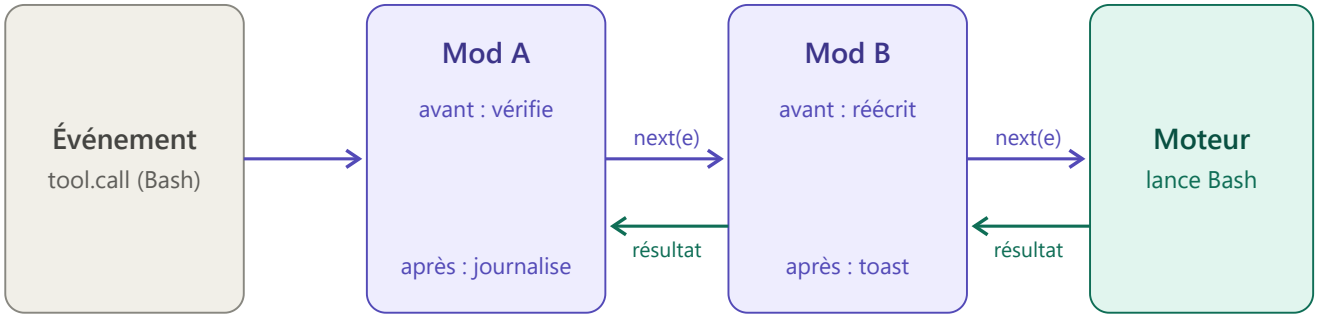


Figure 2 — `next(e)` descend la chaîne jusqu'au moteur ; le résultat remonte.

Un hook a donc quatre façons de répondre :

Ce que fait le hook	Code	Effet
Observer	`return next(e)`	tout se passe normalement
Modifier	`return next({ ...e, command: 'ls -la' })`	la suite voit la version modifiée
Réagir au résultat	`const r = await next(e); …; return r`	agir après coup
Remplacer / bloquer	`return { deny: 'interdit' }`	la suite n'est jamais appelée

> **Oublier `next`** est l'erreur la plus courante. Si votre hook ne l'appelle pas, le moteur ne fait pas son travail habituel : une bande disparaît, un outil n'est jamais lancé… Sauf intention contraire, terminez toujours par `return next(e)`.

Les surfaces

Le même mod tourne partout. L'endroit où il dessine s'appelle la **surface** : `e.surface` vaut `'terminal'`, `'desktop'`, `'vscode'` ou `'mobile'`. Chaque surface fournit sa propre table d'éléments, obtenue par `$.ui.resolve(e)`. Le terminal ne dessine que du texte coloré ; le bureau sait en plus afficher des images SVG.

Un environnement isolé

Votre module tourne dans un bac à sable, séparé du moteur. Conséquences :

- pas de `window`, `process`, `require`, `setTimeout` : utilisez `$.fs`, `$.process`, `$.clock`… ;
- chaque rechargement relance votre module à neuf : les **variables du module** repartent de zéro (voir chapitre 8 pour garder des valeurs) ;
- un hook qui plante n'arrête pas le moteur : il est simplement ignoré (chapitre 13).

5 Anatomie d'un mod

```
mon-mod/
├─ .claude-plugin/
│  ├─ plugin.json          obligatoire : nom, version, description, réglages
│  ├─ marketplace.json     facultatif : pour publier (chapitre 14)
│  └─ types/               écrit par le moteur, ne pas modifier ni versionner
├─ hooks/
│  ├─ hooks.json           obligatoire : désigne le module
│  ├─ register.tsx         obligatoire : le code
│  ├─ outils.ts            facultatif : d'autres fichiers importés
│  └─ register.test.tsx    facultatif : tests automatiques
├─ types/index.d.ts        si le mod utilise $.state (chapitre 8)
├─ assets/carillon.wav     facultatif : sons, images…
└─ README.md
```

`plugin.json` : le manifeste

Champ	Rôle
`name`	identifiant du mod : minuscules, chiffres et tirets. Sert aussi de nom à vos commandes, de préfixe à vos outils et de clé de stockage.
`version`	version au format `majeur.mineur.correctif`. **À augmenter à chaque publication**, sinon les mises à jour ne voient rien.
`description`	une phrase affichée dans les listes de plugins.
`userConfig`	les réglages proposés à l'utilisateur (chapitre 11).
`types`	chemin du contrat de types (`"./types/index.d.ts"`) quand le mod utilise `$.state`.
`dependencies`	autres plugins dont ce mod réutilise les types.

`hooks.json`

Une seule ligne, un seul module, chemin relatif au fichier : `{ "modules": ["./register.tsx"] }`.

Le module de hooks

- Extensions acceptées : `.ts`, `.tsx`, `.js`, `.jsx`, `.mjs`, `.cjs`, `.mts`, `.cts`. Utilisez `.tsx` dès que vous dessinez (JSX).
- Il exporte une fonction `register(on, options)`. `on` accroche les hooks ; `options` contient les réglages de l'utilisateur.
- Il peut importer d'autres fichiers du mod avec `import … from './outils'`. Les imports dynamiques `import()` sont interdits : un module qui en contient ne se charge pas.
- `import … from 'claude-code'` n'apporte que des **types** et quelques fonctions utilitaires (`atom`, `read`, `update` …).

Les types : votre documentation intégrée

Dès que le moteur charge votre mod depuis son dossier, il écrit dans `.claude-plugin/types/` :

- `claude-code/index.d.ts` : toute l'API (environ 14 000 lignes commentées, un exemple par fonction) ;
- `claude-code-tools/index.d.ts` : les outils intégrés (Bash, Edit, Read…) et leurs paramètres ;
- `claude-code-mcp/index.d.ts` : les outils MCP connectés ;
- un `tsconfig.json` que celui de votre mod peut étendre.

Votre éditeur les utilise automatiquement : survolez `$.ui.toast` pour lire sa documentation. Ce dossier contient son propre `.gitignore` : il ne part pas dans votre dépôt.

6 Les événements

Voici le déroulement d'une session et les événements que vous rencontrerez le plus souvent.

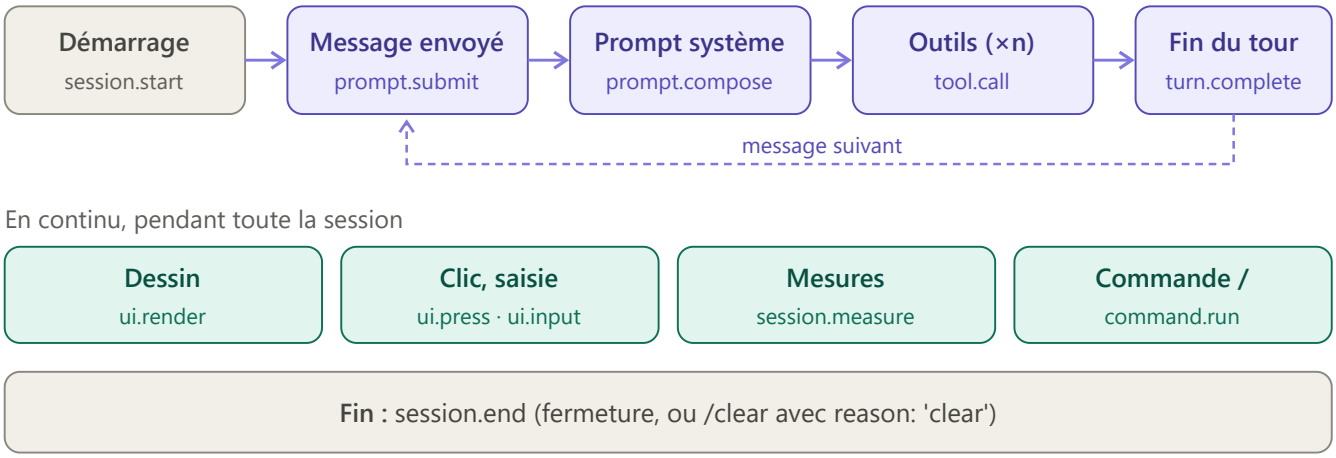


Figure 3 — Les événements dans la vie d'une session.

Événement	Quand	Ce qu'on en fait souvent
`session.start`	une fois, quand la session est prête	enregistrer commandes et outils, ouvrir un panneau, lancer une minuterie
`session.end`	fermeture ou `/clear`	sauvegarder, nettoyer
`prompt.submit`	vous envoyez un message	le corriger, le compléter, le bloquer
`prompt.compose`	le moteur assemble le prompt système	ajouter une section de consignes
`tool.call`	Claude appelle un outil	garde-fou, réécriture, journal, répondre à vos propres outils
`turn.complete`	Claude a fini de répondre	toast, son, notification, statistiques
`session.measure`	contexte, limites ou coût ont changé	redessiner une jauge
`ui.render`	une zone doit être dessinée	bande, panneau, ligne de conversation
`ui.press`, `ui.input`, `ui.select`	clic, saisie, choix dans vos éléments	généralement géré par `onPress`, `onSubmit`…
`command.run`	une commande `/…` est lancée	répondre à vos commandes
`session.append`	une ligne est ajoutée à la conversation	masquer des données sensibles
`classic.Stop`, `classic.SessionEnd`…	les hooks « classiques » de `settings.json`	réutiliser leurs données

La liste complète (agents, skills, télémétrie, flux `turn.step`…) est dans le fichier de types : cherchez `interface Events` ou le nom d'un événement entre apostrophes.

Le bon événement pour le bon besoin

Avant d'écrire, posez-vous deux questions : **quel événement** correspond à ce moment, et **de quoi ai-je besoin dans `$`** ? Par exemple, « prévenir quand une longue réponse se termine » = `turn.complete` + `$.ui.notify` ; « empêcher Claude de modifier `.env` » = `tool.call` filtré sur `Edit` + `{ deny }`.

7 Afficher quelque chose

Les affichages simples (une ligne de code)

Ces appels marchent depuis n'importe quel hook, sans JSX :

Affichage	Code	Remarques
Toast	`$.ui.toast('Enregistré')`	message bref qui disparaît seul
Ligne d'état	`$.ui.status('3 tests échouent')`	`$.ui.status(undefined)` l'efface
Notification	`$.ui.notify('Fini', { title: 'Claude' })`	notification du système, selon les préférences de l'utilisateur
Ligne de journal	`$.ui.log('…')`	ligne discrète dans la conversation
Presse-papiers	`$.ui.copy({ text })`	renvoie `{ isCopied }`
Son	`$.audio.play({ asset: 'assets/ding.wav' })`	fichier du mod ; voir aussi chapitre 15

Dessiner avec `ui.render`

Pour tout ce qui est plus riche, on accroche un hook à `ui.render` en ciblant un **composant** :

- `{ component: 'AbovePrompt' }` : la bande au-dessus de la zone de saisie ;
- `{ component: 'Pane', requestId: 'mon-id' }` : un panneau que vous avez ouvert ;
- d'autres composants de la conversation (`UserMessage`, `CommandOutput`…), pour les redessiner.

Le hook reçoit dans `e` : `e.surface` (où l'on dessine), `e.props` (les données du composant, dont `bodyColumns`, la largeur disponible en colonnes) et `e.viewport` (taille de la zone). Il renvoie un **arbre d'éléments**.

Les éléments

On ne les importe pas : on les demande à la surface, puis on les utilise en JSX.

```
const { Box, Text, Button } = $.ui.resolve(e)
```

Élément	Rôle	Disponible
`Box`	conteneur (disposition en ligne ou colonne, marges, bordure, couleur de fond)	partout
`Text`	texte : `color` (une clé du thème comme `'success'`, `'error'`, `'warning'`, ou une couleur brute), `bold`, `dimColor`…	partout
`Button`	bouton cliquable (`onPress`, `hotkey`, `variant`)	partout
`Input`, `Select`	champ de saisie, liste de choix	pas sur mobile
`Markdown`	texte mis en forme comme une réponse de Claude	partout
`Code`, `Link`	bloc de code (ou diff), lien	partout
`Svg`	image vectorielle (graphiques, cartes)	pas dans le terminal
`Raster`, `Image`	grille de cases colorées, image	terminal

La liste exacte des éléments et de leurs propriétés est dans les types (cherchez `Elements`). Un arbre qui utilise un élément ou une propriété inconnus de la surface est refusé, et le hook est ignoré.

Exemple : une bande au-dessus du prompt

```
import type { Register } from 'claude-code'

export const register: Register = on => {
  // Redessiner quand une mesure change (après chaque réponse).
  on('session.measure', ($, e, next) => {
    $.ui.invalidate('ui.render')
    return next(e)
  })

  on('ui.render', { component: 'AbovePrompt' }, async ($, e, next) => {
    const { Box, Text } = $.ui.resolve(e)
    const { context } = await $.session.usage()
    const autres = await next(e)        // les bandes des autres mods
    const pct = context?.percent              // vide avant la 1re réponse
    const couleur = pct === undefined ? 'subtle' : pct > 80 ? 'error' : 'success'
    return (
      <Box flexDirection="column">
        {autres}
        <Text color={couleur}>Contexte : {pct ?? '—'} %</Text>
      </Box>
    )
  })
}
```

> **Pourquoi `await next(e)` ?** La bande est partagée : plusieurs mods peuvent y dessiner. En récupérant le dessin des autres et en l'incluant dans votre `Box`, vous les gardez à l'écran. Si vous n'avez rien à montrer, renvoyez simplement `next(e)`.

Redessiner

Le moteur ne redessine pas votre bande de lui-même quand *vos* données changent. Deux solutions :

- appeler `$.ui.invalidate('ui.render')` quand vous savez que quelque chose a changé (comme ci-dessus) ;
- mieux : lire vos données avec `$.state` pendant le dessin. Toute écriture redessine alors automatiquement ce qui les lit (chapitre 8).

Ouvrir un panneau latéral

Un panneau se fait en deux temps : l'**ouvrir** (avec un identifiant), puis le **dessiner** dans un hook qui cible cet identifiant.

```
const PANNEAU = 'notes'

on('session.start', async ($, e, next) => {
  void $.ui.open({ id: PANNEAU, title: 'Mes notes' })
  return next(e)
})

on('ui.render', { component: 'Pane', requestId: PANNEAU }, async ($, e) => {
  const { Box, Text } = $.ui.resolve(e)
  const largeur = e.props.bodyColumns        // en colonnes de texte
  return (
    <Box flexDirection="column">
      <Text bold>Bonjour !</Text>
      <Text dimColor>Largeur disponible : {largeur} colonnes</Text>
    </Box>
  )
})
```

`$.ui.open` renvoie `{ isPlaced }` : vrai si une surface l'affiche maintenant. Dans le terminal en plein écran ou sur le bureau, le panneau se place à côté de la conversation. Une commande `/notes` qui le rouvre est une bonne idée : l'utilisateur peut l'avoir fermé (chapitre 10).

Boutons et interactions

```
const { Box, Text, Button } = $.ui.resolve(e)
return (
  <Box flexDirection="row" gap={1}>
    <Button key="plus" hotkey="p" variant="primary"
            onPress={() => update($, compteur, n => n + 1)}>Ajouter</Button>
    <Button key="zero" onPress={() => update($, compteur, () => 0)}>Remettre à zéro</Button>
  </Box>
)
```

- Donnez à chaque bouton une `key` **unique** : c'est ainsi que la surface le retrouve après un redessin.
- Le gestionnaire `onPress` reste dans votre mod ; le clic déclenche l'événement `ui.press`.
- Dans le terminal, `hotkey` (un chiffre ou une lettre) active le bouton quand la zone a le focus ; Échap revient à la saisie.
- Un hook de dessin **n'écrit jamais** de données : il lit. Les écritures se font dans les gestionnaires (`onPress`) ou dans d'autres événements.

Adapter le dessin à la surface

```
if (e.surface !== 'terminal') {
  const { Svg } = $.ui.resolve(e)
  return <Svg source={carteSvg(donnees)} alt="Jauge du contexte" />
}
// Terminal : une version texte.
const { Text } = $.ui.resolve(e)
return <Text>[#####-----] 50 %</Text>
```

> **Piège vécu.** La table d'éléments du terminal contient parfois `Svg` sans savoir le dessiner. Testez `e.surface !== 'terminal'`, jamais `'Svg' in table`, sinon le terminal n'affiche rien.

8 Garder des données

Trois endroits possibles, qui ne durent pas aussi longtemps :

Où	Durée de vie	Redessine ?	Usage
Variable du module (`let x`)	jusqu'au prochain rechargement du mod	non	cache, valeur calculée une fois
`$.state`	la session (survit aux rechargements)	**oui, automatiquement**	ce que l'interface affiche
`$.store`	toutes les sessions (fichier JSON)	non	préférences, historique, dernière mesure

`$.store` : la mémoire permanente

```
const total = Number((await $.store.get('tours')) ?? 0) + 1
await $.store.set('tours', total)
// aussi : $.store.delete(clé), $.store.keys()
```

Les valeurs doivent être du JSON (une `Date` devient une chaîne, une `Map` devient `{}`). Limite : 4 Mo au total par mod.

`$.state` : l'état qui redessine

`$.state` garde des valeurs pour la durée de la session. Toute lecture faite **pendant un dessin** abonne ce dessin : quand la valeur change, il est redessiné tout seul. C'est la bonne façon d'alimenter une bande ou un panneau.

1. Déclarer le contrat de types

Chaque valeur se déclare dans `types/index.d.ts`, sous le nom du mod :

```
// types/index.d.ts
export type Note = { texte: string; date: number }

declare module 'claude-code' {
  interface PluginState {
    'mes-notes': { notes: Shaped<Note[]>; compteur: number }
  }
}
```

...et se référence dans `plugin.json` : `"types": "./types/index.d.ts"`. `claude plugin validate` vérifie ensuite que le code respecte ce contrat.

2. Lire et écrire avec `atom`, `read`, `update`

```
import { atom, read, update } from 'claude-code'
import type { Register } from 'claude-code'

// Une valeur, avec sa valeur initiale.
const compteur = atom({ plugin: 'mes-notes', key: 'compteur' } as const, 0)

export const register: Register = on => {
  on('turn.complete', async ($, e, next) => {
    await update($, compteur, n => n + 1)      // écrit → redessine les lecteurs
    return next(e)
  })

  on('ui.render', { component: 'AbovePrompt' }, async ($, e, next) => {
    const { Box, Text } = $.ui.resolve(e)
    const n = await read($, compteur)              // lit et s'abonne
    return <Box flexDirection="column">{await next(e)}<Text>{n} tours</Text></Box>
  })
}
```

- `update` lit, applique votre fonction et écrit, en réessayant si la valeur a changé entre-temps : deux clics rapides comptent bien pour deux.
- Le `plugin` et la `key` doivent être écrits en **littéraux** (pas calculés) : le moteur les vérifie en lisant le code.
- Seul le mod propriétaire écrit ses valeurs ; les autres peuvent les lire.

3. Survivre aux changements de format

Comme `$.state` survit aux rechargements, votre nouveau code peut lire une valeur écrite par l'ancien. Pour une valeur déclarée `Shaped<T>`, ajoutez une étiquette de format et changez-la quand la forme des données change :

```
const notes = atom({ plugin: 'mes-notes', key: 'notes' } as const, [], { shape: 'notes-2' })
// Une valeur écrite avec une autre étiquette est lue comme la valeur initiale [].
```

Combiner les deux

Pour garder une valeur affichée d'une session à l'autre : écrivez-la dans `$.state` *et* dans `$.store`, et au `session.start`, rechargez `$.state` depuis `$.store`. C'est ce que font les mods usage-limits et usage-forecast pour montrer la dernière mesure connue avant la première réponse d'une session.

9 Agir sur Claude

Modifier un message avant envoi : `prompt.submit`

```
on('prompt.submit', ($, e, next) => {
  // Ajoute une consigne à chaque message qui parle de tests.
  if (/\btests?\b/i.test(e.text))
    return next({ ...e, text: e.text + '\n\n(Utilise vitest, pas jest.)' })
  return next(e)
})
```

Un hook peut aussi refuser le message (`{ drop: 'raison' }`). Les commandes `/…` passent par cet événement mais leur texte ne se réécrit pas ici : utilisez `command.run`.

Ajouter des consignes permanentes : `prompt.compose`

Le prompt système est une liste de **sections** (`{ id, text, scope }`). Ajoutez la vôtre à la fin de ce que renvoie `next(e)` :

```
on('prompt.compose', async ($, e, next) => {
  const { sections } = await next(e)
  return {
    sections: [...sections, {
      id: 'mon-mod:style',                      // toujours préfixé par le nom du mod
      text: 'Réponds toujours en français. Commente le code en français.',
      scope: 'session',
    }],
  }
})
```

> **Règle importante : ce qui a été envoyé est figé.** Le modèle met en cache le début de chaque conversation (prompt système, outils, messages précédents). Si votre section change en cours de conversation, ce cache est perdu (réponses plus lentes et plus coûteuses) et le raisonnement précédent aussi. Une section doit donc donner **le même texte pendant toute une conversation**. Pour apporter une information nouvelle en cours de route, ajoutez plutôt une ligne à la fin avec `$.session.append({ message: { type: 'user', content: [{ type: 'text', text: '…' }] } })`.

Surveiller les outils : `tool.call`

C'est l'événement le plus puissant. Chaque fois que Claude veut lire un fichier, l'éditer, lancer une commande…, votre hook voit l'appel et ses paramètres.

Un garde-fou

```
const PROTEGE = /(^|\/)\.env(\.|$)/

on('tool.call', { tool: 'Edit' }, ($, e, next) =>
  PROTEGE.test(e.file_path)
    ? { deny: `${$.plugin.name} : ${e.file_path} est protégé.` }
    : next(e),
).catch(($, e, next) =>
  // Si le hook plante, on refuse par prudence (sauf si next a déjà tourné).
  next.called ? next(e) : { deny: `${$.plugin.name} : le garde-fou a échoué.` },
)
```

Claude reçoit la raison du refus et s'adapte. Le `.catch` est essentiel pour un garde-fou : sans lui, un hook qui plante est ignoré… et l'action interdite passe.

Modifier ou observer

```
on('tool.call', { tool: 'Bash' }, async ($, e, next) => {
  const r = await next({ ...e, command: e.command.trim() })   // réécrit puis exécute
  const echec = r.deny === undefined && r.isError === true
  $.ui.status(echec ? `échec : ${e.command.slice(0, 40)}` : undefined)
  return r
})
```

Donner un nouvel outil à Claude

Un mod peut déclarer un outil que Claude appellera de lui-même. On le déclare au démarrage, et on y répond dans `tool.call`. L'outil s'appelle `mcp__<nom-du-mod>__<nom>`.

```
on('session.start', async ($, e, next) => {
  await $.tool.register({
    name: 'heure',
    description: "Donne l'heure locale de l'utilisateur.",
    inputSchema: { type: 'object', properties: {} },
    isDeferred: false,           // visible d'emblée par Claude
  })
  return next(e)
})

on('tool.call', { tool: 'mcp__mon-mod__heure' }, async $ => {
  const maintenant = new Date(await $.clock.now())
  return { result: maintenant.toLocaleTimeString('fr-FR') }
})
```

Sans `isDeferred: false`, l'outil est rangé derrière la recherche d'outils : Claude ne voit que son nom et charge sa description quand il en a besoin.

Appeler le modèle soi-même

- `$.model.complete({ model, prompt })` : une question isolée, sans historique (résumer, classer…). Le résultat dit toujours s'il a répondu (`isAnswered`) et combien il a coûté (`usage`).
- `$.model.fork({ prompt })` : une question posée sur toute la conversation en cours, en profitant de son cache.
- `$.prompt.submit({ text })` : envoie un message comme si l'utilisateur l'avait tapé, au prochain moment libre.
- `$.agent.register` / `$.agent.spawn` : déclarer et lancer des sous-agents.

Ces appels sont facturés sur le compte de l'utilisateur : utilisez-les avec mesure.

10 Commandes, minuteries, système

Ajouter une commande `/mon-mod`

```
on('session.start', async ($, e, next) => {
  await $.command.register({ name: 'notes', description: 'Ouvre le panneau des notes' })
  return next(e)
})

on('command.run', { command: 'notes' }, async ($, e) => {
  await $.ui.open({ id: 'notes', title: 'Mes notes' })
  return { text: 'Panneau des notes ouvert.' }   // ligne affichée (et lue par Claude)
})
```

La réponse `{ text }` apparaît dans la conversation ; ajoutez `context` pour des notes que seul Claude lit. Les arguments tapés après la commande sont dans `e.args`.

Minuteries : `$.clock`

Appel	Effet
`await $.clock.now()`	l'heure, en millisecondes
`await $.clock.sleep(500, { signal: next.signal })`	attendre (compte dans le temps du hook)
`$.clock.after(ms, fn)`	appeler `fn` une fois plus tard
`$.clock.every(ms, fn)`	appeler `fn` régulièrement

Les deux dernières renvoient un objet permettant d'annuler. Lancez les tâches de fond dans `session.start` : un rechargement du mod les arrête toutes, et `session.start` les relance.

```
on('session.start', ($, e, next) => {
  $.clock.every(60_000, () => { $.ui.invalidate('ui.render') })   // redessine chaque minute
  return next(e)
})
```

Fichiers, processus, réseau

Besoin	Appel
Lire un fichier	`await $.fs.read(chemin)` (texte) ou `{ as: 'bytes' }` (base64)
Écrire, lister, inspecter	`$.fs.write`, `$.fs.list`, `$.fs.stat`
Lancer une commande	`await $.process.run({ argv: ['git', 'status'] })` → `{ exitCode, stdout, stderr }`
Suivre une commande longue	`$.process.spawn({ argv })` (flux)
Appeler un site web	`await $.http.fetch(url, { method, headers, body })`
Dossier de la session	`await $.session.root()`
Mesures (contexte, limites, coût)	`await $.session.usage()`
Version du moteur	`await $.session.version()`

`$.process.run` exécute la commande avec les droits de l'utilisateur, en lui passant les arguments un par un (`argv`) : pas d'interprétation par un shell, donc pas d'injection par des espaces ou des guillemets.

Travail qui dure : le budget

Chaque hook dispose d'un **budget de temps**. Les appels à `next` et à `$` en cours ne comptent pas, mais `$.clock.sleep` si. `next.signal` s'annule quand l'événement est abandonné (l'utilisateur interrompt, le budget est épuisé) : passez-le à tout ce qui peut attendre. Un travail qui doit durer au-delà d'un événement se lance depuis `session.start` et se poursuit avec `$.clock.every` / `after`.

11 Les réglages utilisateur

Un mod peut proposer des réglages. Ils se déclarent dans `plugin.json` sous `userConfig` :

```
{
  "name": "usage-limits",
  "version": "0.2.2",
  "description": "Affiche les limites d'usage sur 5 heures et sur 7 jours",
  "userConfig": {
    "theme": {
      "type": "string",
      "title": "Thème de la carte",
      "description": "auto suit le thème de l'affichage.",
      "default": "auto",
      "options": ["auto", "sombre", "clair"]
    }
  }
}
```

- Types possibles : `string`, `number`, `boolean`, liste de chaînes. Un champ `string` avec `options` devient une **liste de choix** ; une valeur hors liste compte comme non définie (la valeur par défaut s'applique).
- Chaque réglage non secret apparaît dans `/config`. Le changer recharge le mod avec les nouvelles valeurs.
- Les valeurs sont stockées dans `~/.claude/settings.json`, sous `pluginConfigs["<mod>@<marketplace>"].options` (ou `"<mod>"` pour un mod chargé par `--plugin-dir`). Les champs marqués sensibles vont dans le stockage sécurisé.
- Un champ obligatoire sans valeur empêche le mod de se charger, avec un message qui le nomme.

Lire les réglages

```
export const register: Register = (on, options) => {
  const theme = String(options.theme ?? 'auto')   // valeurs par défaut déjà appliquées
  …
}
```

> **Réglages ou `$.store` ?** Les `userConfig` sont ce que l'utilisateur choisit explicitement (thème, seuil, activation d'un son). `$.store` est ce que le mod mémorise de lui-même (dernière mesure, historique). Ne mélangez pas les deux.

12 Tester son mod

Niveau 1 : la validation

```
claude plugin validate mon-mod
```

Elle lit le manifeste, le module et, s'il existe, le `marketplace.json`. Elle liste :

- les événements branchés et les appels faits sur `$` ;
- les clés `$.state` utilisées et leur conformité au contrat ;
- les hooks « décisifs » (qui peuvent refuser quelque chose), avec ou sans `.catch` ;
- tout ce que le moteur refuserait au chargement.

Niveau 2 : les types

Une fois le mod chargé une fois, ses types sont posés : `tsc -p mon-mod` vérifie tout le code (avec un `tsconfig.json` à la racine du mod qui étend celui de `.claude-plugin/types/`). L'éditeur fait la même chose en continu.

Niveau 3 : les tests automatiques

```
claude plugin test mon-mod
```

Cette commande exécute les fichiers `*.test.ts` / `*.test.tsx` du mod **avec le vrai moteur**, mais sans session : c'est le test qui joue le rôle du moteur.

Le principe

Un test reçoit `$` et `on`. Avec `$`, il **déclenche** des événements (`$.turn.complete(…)`, `$.tool.call(…)`). Avec `on`, il installe des hooks qui se placent **sous** votre mod et **remplacent le moteur** : ils décident de ce que répond `$.store.get`, `$.session.usage`, `$.clock.now`…, et enregistrent ce que votre mod a demandé (un toast, une notification).

```
import { expect, test } from 'claude-code/testing'

test('un tour long affiche un toast avec la durée', async ($, on) => {
  const vus: string[] = []
  // Ce que le « moteur » répond :
  on('turn.complete', () => ({ text: 'ok' }))
  on('ui.toast', (_, e) => { vus.push(JSON.stringify(e)); return { value: undefined } })

  // On déclenche l'événement :
  await $.turn.complete({
    answer: 'ok', isAborted: false, turnId: 't1', reason: 'answer', durationMs: 12_000,
  })

  expect(vus.some(v => v.includes('12 s'))).toBe(true)
})
```

Un appel sur `$` (comme `ui.toast`) se simule en répondant `{ value: … }` ; un événement (comme `turn.complete`), par son propre résultat.

Tester un dessin, sur toutes les surfaces

```
for (const surface of ['terminal', 'desktop'] as const) {
  const ui = await $.ui.mount({
    plugin: 'mon-mod', surface, component: 'AbovePrompt',
    props: { bodyColumns: 80, view: {} },
  })
  const ligne = await ui.find({ type: 'Text', text: /Contexte/ })
  expect(ligne?.text).toBe('Contexte : 42 %')
  await ui.unmount()
}
```

`ui.find` cherche un élément, `ui.press` clique sur un bouton par sa `key`, `ui.input` remplit un champ. Boucler sur les surfaces prouve que le mod ne dépend pas de l'une d'elles. Les propriétés exactes à passer à `mount` pour chaque composant sont dans les types.

> **Piège.** Comme rien n'existe sous votre mod pendant un test, **tout** ce qu'il appelle doit être simulé : si votre bande appelle `next(e)`, il faut un hook `ui.render` de test qui renvoie quelque chose ; si elle lit `$.store`, un hook `store.get`, etc.

Tester les réglages et l'horloge

- `test('nom', { options: { theme: 'clair' } }, async ($, on) => …)` donne des réglages au mod.
- `mock.clock(on)` (importé de `claude-code/testing`) fournit une horloge que le test fait avancer.

13 Déboguer

Ce qui se passe quand un hook échoue

Un hook qui lève une exception, dépasse son budget ou renvoie une forme invalide est **ignoré seul** : la chaîne continue sans lui et l'application ne plante pas. C'est rassurant pour l'utilisateur, mais déroutant pour le développeur : un mod qui « ne fait rien » a en général été ignoré pour une raison précise.

Où lire la raison

Situation	Où regarder
Mod chargé depuis son dossier (`--plugin-dir`, `CLAUDE_CODE_PLUGIN_DIRS`, mod écrit par Claude)	une **ligne grise** dans la conversation, une fois par chargement : `mon-mod: ui.render hook skipped: …`
Mod installé depuis un marketplace	le journal : lancez `claude --debug`
Le module ne se charge pas du tout	ligne grise ou journal ; `claude plugin validate` dit presque toujours pourquoi

Les causes les plus fréquentes

- **Un élément ou une propriété inconnus** de la surface (un `Svg` dans le terminal, une `borderStyle` inventée…). Le message cite l'élément et la raison.
- **Un `next` oublié** : rien n'est cassé, mais ce que le moteur aurait fait n'a pas lieu.
- **Une valeur absente** au début de session : `$.session.usage()` est vide avant la première réponse. Prévoyez les `undefined`.
- **Une écriture pendant un dessin** : `$.state.set` est refusé dans `ui.render`.
- **Un autre mod** placé au-dessus qui court-circuite le vôtre : le journal le signale (`bypassed by …`).

Afficher ses propres traces

Pas de `console.log` visible : utilisez `$.ui.log('valeur : ' + JSON.stringify(x))` pour une ligne dans la conversation, ou `$.ui.status(…)` pour une valeur qui change souvent. Retirez-les avant de publier.

Rechargement

- Un enregistrement qui ne change pas le contenu d'un fichier ne recharge rien.
- Après un rechargement raté, la sauvegarde suivante réessaie.
- Un rechargement relance `register` puis `session.start` : les variables du module repartent de zéro, `$.state` et `$.store` restent.
- Pour un mod installé dont le marketplace est un dossier local, `/reload-plugins` relit ce dossier.

14 Publier et mettre à jour

Le principe : un dépôt GitHub qui est un « marketplace »

Un marketplace est simplement un dépôt contenant `.claude-plugin/marketplace.json`, qui liste un ou plusieurs mods. Deux organisations possibles :

Un mod par dépôt

```
mon-mod/                        ← racine du dépôt
├─ .claude-plugin/
│  ├─ plugin.json
│  └─ marketplace.json
├─ hooks/…
└─ README.md
```

```
{
  "name": "mon-mod",
  "owner": { "name": "Moi" },
  "plugins": [
    { "name": "mon-mod", "source": "./" }
  ]
}
```

Plusieurs mods (comme teddy-mods)

```
mes-mods/                       ← racine du dépôt
├─ .claude-plugin/
│  └─ marketplace.json
├─ mod-a/    (.claude-plugin/, hooks/)
├─ mod-b/
└─ README.md
```

```
{
  "name": "mes-mods",
  "owner": { "name": "Moi" },
  "plugins": [
    { "name": "mod-a", "source": "./mod-a",
      "description": "…" },
    { "name": "mod-b", "source": "./mod-b" }
  ]
}
```

Avant de publier

1. `claude plugin validate` et `claude plugin test` passent.
2. **Aucun secret** dans les fichiers : clé d'API, jeton, adresse interne, données personnelles. Relisez tout, fichiers cachés compris.
3. Le dossier `.claude-plugin/types/` est exclu (il a son propre `.gitignore`) ; ajoutez au `.gitignore` les fichiers de votre éditeur et les journaux.
4. Choisissez la visibilité : **public** (tout le monde peut lire le code et installer le mod) ou **privé** (seules les personnes invitées sur GitHub, qui doivent aussi avoir configuré GitHub sur leur machine).
5. Un `README.md` qui explique ce que fait le mod, avec une capture d'écran et les lignes d'installation ci-dessous.

Installer un mod publié

Dans une session Claude Code (terminal) :

```
/plugin install mon-mod --marketplace compte/depot
```

Claude Code demande d'ajouter le marketplace (`y`), puis la portée (utilisateur par défaut), puis les réglages éventuels. Le mod est actif aussitôt.

Depuis un shell :

```
claude plugin install mon-mod --marketplace compte/depot
```

Dans l'application de bureau (où `/plugin` n'existe pas) : bouton **+** à côté de la zone de saisie › **Plugins** › **Add marketplace** (`compte/depot`) › **Add plugin**.

> **Dépôt privé.** L'installation ne demande jamais d'identifiants : la machine doit déjà avoir accès à GitHub (clé SSH, ou `gh auth login` puis `gh auth setup-git`).

Publier une nouvelle version

1. Modifiez le mod et **augmentez** `version` dans son `plugin.json` (`0.2.1` → `0.2.2`). Sans cela, la mise à jour ne voit aucun changement.
2. Validez, testez, commitez, poussez.
3. Chez chaque utilisateur :

   ```
   claude plugin marketplace update depot
   claude plugin update mon-mod@depot
   ```

   puis redémarrer l'application (ou `/reload-plugins` dans une session).

Où les mods sont installés

Les mods s'installent **machine par machine**, dans le dossier `~/.claude` de l'utilisateur. Une session lancée par SSH sur un serveur lit le `~/.claude` *du serveur* : il faut y installer les mods aussi. Un mod installé au niveau utilisateur depuis un terminal se charge aussi dans les sessions locales de l'application de bureau.

15 Pièges et bonnes pratiques

Ces conseils viennent de l'écriture des quatre mods du dépôt teddy-mods.

Conception

- **Commencez petit.** Un toast d'abord, puis une bande texte, puis la version graphique. Validez à chaque étape.
- **Pensez terminal.** Prévoyez toujours une version texte : certains utilisateurs ne verront jamais votre SVG.
- **Ne volez pas la place.** Une bande doit tenir sur une ou deux lignes ; un panneau ne s'ouvre de lui-même que là où il se place à côté de la conversation (`e.viewport.isFullscreen` , ou la `presentation` de `command.run`).
- **Restez discret.** Un toast à chaque tour devient vite pénible : ajoutez un seuil (turn-alert ne prévient qu'au-delà de 60 s) et des réglages pour couper son et notification.

Dessin

- **Largeur d'un panneau** : `e.props.bodyColumns` la donne en colonnes de texte. Sur le bureau, une colonne vaut environ 7 px : utile pour dimensionner un SVG.
- **Cartes SVG** : elles sont affichées comme des images. Le texte n'y revient pas à la ligne tout seul (découpez en `<tspan>`). Préfixez les `id` (dégradés, motifs) avec le nom du mod, sinon deux cartes se mélangent.
- **Thème clair et sombre** : dans un SVG, définissez les couleurs par des variables CSS et redéfinissez-les sous `@media (prefers-color-scheme: light)` : la carte suit alors le thème de l'application. Proposez un réglage `auto / sombre / clair` .
- **Boutons** : une `key` unique et stable pour chacun.

Données

- **Début de session** : `$.session.usage()` est vide avant la première réponse. Enregistrez la dernière mesure dans `$.store` et affichez-la (en le signalant : « dernière mesure ») en attendant.
- **Le stockage dépend du mode d'installation.** Un mod chargé depuis son dossier et le même mod installé depuis un marketplace n'ont pas le même `$.store` : passer de l'un à l'autre repart de zéro.
- **Rien d'important dans les variables du module** : elles disparaissent à chaque rechargement.
- **Changez l'étiquette** `shape` quand la forme d'une valeur de `$.state` change.

Système

- **Son sous Windows** : `$.audio.play` peut ne trouver aucun lecteur. turn-alert fabrique un WAV en mémoire et le joue avec PowerShell (`$.process.run` + `System.Media.SoundPlayer`), avec `$.audio.play` en secours.
- **Un mod, une source.** Ne le chargez pas à la fois par `CLAUDE_CODE_PLUGIN_DIRS` et par un marketplace : il s'exécuterait deux fois (deux bandes, deux toasts).
- **Application de bureau sous Windows (MSIX)** : son dossier de données réel est sous `AppData\Local\Packages\Claude_…\LocalCache\Roaming\Claude` , pas directement sous `AppData\Roaming` .

Sécurité

- **Un garde-fou a toujours un** `.catch` qui refuse en cas d'erreur.
- **Chemins** : pour autoriser seulement un dossier, comparez le chemin réel (`$.fs.stat(chemin, { resolve: true })` donne `realPath` , liens résolus) à une liste blanche. Une liste noire de noms est contournable.
- **Commandes** : passez toujours les arguments séparément dans `argv` , jamais une chaîne construite à la main.
- **Ne publiez jamais de secret**, même dans un fichier de test.

Versions

- Augmentez `version` à chaque publication.

- L'API étant en accès anticipé, relisez les types après une mise à jour de Claude Code et relancez `validate` et `test`.

16 Aide-mémoire et références

Les commandes

Commande	Rôle
`claude plugin validate <dossier>`	vérifier manifeste, code et marketplace
`claude plugin test <dossier>`	lancer les tests du mod
`claude --plugin-dir <dossier>`	session avec le mod chargé depuis son dossier
`claude --debug`	journal détaillé
`claude plugin install <mod> --marketplace <compte/depot>`	installer
`claude plugin marketplace update <depot>`	relire un marketplace
`claude plugin update <mod>@<depot>`	mettre à jour un mod
`claude plugin list`	mods installés et leur origine
`/reload-plugins`	recharger les plugins dans une session
`/config`	changer les réglages des mods

Les réponses d'un hook

Je veux…	J'écris…
laisser faire	`return next(e)`
modifier l'entrée	`return next({ ...e, champ: valeur })`
agir après	`const r = await next(e); … ; return r`
bloquer un outil	`return { deny: 'raison' }`
répondre à mon outil	`return { result: … }`
répondre à ma commande	`return { text: '…' }`
dessiner	`return <Box>…</Box>`
protéger un garde-fou	`on(…).catch(($, e, next) => next.called ? next(e) : { deny: '…' })`

Les noms de `$` les plus utiles

Nom	Contenu
`$.ui`	`resolve`, `toast`, `status`, `notify`, `log`, `open`, `invalidate`, `copy`, `focus`, `view`, `selection`
`$.session`	`usage`, `root`, `version`, `messages`, `append`, `surfaces`
`$.state` / `$.store`	état de session / mémoire permanente
`$.tool`, `$.command`, `$.agent`	déclarer et appeler outils, commandes, sous-agents
`$.prompt`	`submit`, `fill`, `read`, `paint` (la zone de saisie)
`$.model`	`complete`, `fork`
`$.clock`	`now`, `sleep`, `after`, `every`
`$.fs`, `$.process`, `$.http`, `$.audio`	fichiers, commandes, réseau, son
`$.plugin`	informations sur votre mod (`name`…)

Glossaire

Moteur
le cœur de Claude Code, commun au terminal, au bureau, à VS Code et au mobile.

Événement
un moment où le moteur demande l'avis des mods (`tool.call`, `ui.render`…).

Hook
une fonction `($, e, next)` accrochée à un événement.

Chaîne
la suite des hooks de tous les mods pour un même événement, terminée par le moteur.

Filtre (matcher)
le deuxième argument de `on`, qui restreint le hook à certains cas.

Surface
là où l'on dessine : `terminal`, `desktop`, `vscode`, `mobile`.

Composant
une zone dessinable : `AbovePrompt` (bande), `Pane` (panneau), lignes de conversation…

Marketplace
un dépôt (ou dossier) qui liste des plugins installables.

Contrat de types
le fichier `types/index.d.ts` qui déclare les valeurs `$.state` d'un mod.

La référence qui fait foi

> **Le fichier de types.** Après le premier chargement d'un mod depuis son dossier, ouvrez `.claude-plugin/types/claude-code/index.d.ts`. Cherchez le nom d'un événement entre apostrophes (`'tool.call'`), d'un élément (`Pane: {`) ou d'un type (`ToolCallResult`) : vous y trouverez ses entrées, ses résultats, sa documentation et un exemple. Ce fichier correspond exactement à *votre* version de Claude Code ; ce guide, lui, peut vieillir.

- **Le skill `plugin-authoring`** de Claude Code : demandez à Claude de faire ou de corriger un mod, il le charge avec ses exemples complets (bande, panneau, garde-fou, tests).

- **Le dépôt Teddy57600/teddy-mods** : quatre mods réels et testés, à lire comme exemples (bande SVG avec thème, panneau qui s'adapte à la largeur, alerte sonore, tests).
- **Le guide de deux pages** (`docs/guide-mods-claude-code.pdf`) : l'essentiel, pour ceux qui connaissent déjà le système.

Guide complet des mods Claude Code — rédigé à partir de Claude Code 2.1.317, de sa référence pour les auteurs de mods et des mods du dépôt Teddy57600/teddy-mods. Novembre 2026. L'API est en accès anticipé et peut évoluer.